

Facharbeit



Thema: Offline-Time-Measuring im internen Projektmanagement-Tool
Hersteller: Raphael Waltert
Datum: 24. Februar 2005

Inhaltsverzeichnis

1	Umfeld und Ablauf	3
1.1	Aufgabenstellung	3
1.2	Vorkenntnisse	3
1.3	Vorarbeit	3
1.3.1	Online Zeitmessung	3
1.3.2	Befehle	4
1.3.3	Authentifikation	4
1.3.4	Config Files	5
1.4	Firmenstandarts	5
1.4.1	Filesystem	5
1.4.2	File-Namen	6
1.4.3	Code-Formatierung	6
1.4.4	Namensgebung	7
1.4.5	Extreme-Programming die Wichtigsten Grundsätze	8
1.5	Zeitplan	8
1.6	Arbeitsjournal	9
2	Project	14
2.1	Verfeinerung des Auftrags	14
2.2	Planung	14
2.3	Einführung in das RDPM	14
2.3.1	Was ist RDPM?	14
2.3.2	Simplifiziertes Klassendiagramm des Clients	15
2.3.3	Simplifiziertes Klassendiagramm des Servers	15
2.4	Konzept für das Abspeichern der Time-Measuring Daten	15
2.4.1	Madeleine Variante	16
2.4.2	Marshal Variante	17
2.4.3	YAML Variante	18
2.4.4	Variantenvergleich	18
2.4.5	Entscheid	19
2.5	Realisierung	20
2.5.1	Vorgehensweise	20
2.5.2	Das Abspeichern der Time-Measuring Daten im YAML-Format	20
2.5.3	Das Synchronisieren mit dem Server	21
2.5.4	Zeitmessung für die parallele Arbeit an zwei oder mehr Projekten/Tasks	22
2.6	Testen	22
2.6.1	Testprotokoll	23
2.6.2	Testbericht	24
2.7	Installationsanleitung	24
2.7.1	Client-Installation	24
2.7.2	Server-Installation	25
2.8	Benutzeranleitung	26
2.9	Schlusswort	26
3	Anhang	27
3.1	Literaturverzeichnis	27
3.2	Glossar	27
3.3	Code Anhang	28

1 Umfeld und Ablauf

1.1 Aufgabenstellung

Erweitere den Client des Projektmanagement-Tools RDPM so, dass Zeitmessungen offline vorgenommen werden können. Evaluiere verschiedene Ansätze, wie dazu die Zeitmessungen auf dem Client persistiert werden können und vergleiche sie falls nötig mit Hilfe von Spikes (<http://www.extremeprogramming.org/rules/spike.html>). Entscheide dich für einen Persistenz-Ansatz und begründe deine Entscheidung. Implementiere die gewählte Lösung nach "Test driven Design"-Regeln. Erstelle ausserdem ein Command-Line-Interface, das die Zeitmessung auch für die parallele Arbeit an zwei oder mehr Projekten/Tasks so einfach wie möglich macht:

```
status: no current task
rdpm > start /project1/task2 # Start-Variante 1: kein voreingestellter Task
status: working on /project1/task2
rdpm > start /project1/task3 # Start-Variante 2: laufender Task tritt pausiert in
den Hintergrund
status: working on /project1/task3
rdpm > stop # Stop-Variante 1 : Hintergrund-Task wird wieder aktiviert
status: working on /project1/task2
rdpm > stop # Stop-Variante 2 : letzter akiver Task wird pausiert
status: pausing on /project1/task2
rdpm > start # Start-Variante 3: pausierter Task wird reaktiviert
status: working on /project1/task2
rdpm > start /project1/task3 # Start-Variante 2
status: working on /project1/task3
rdpm > stop all # Stop-Variante 3 : Kaffeepause
status: pausing on /project1/task3
```

Die Dokumentation ist bei Extreme-Programming im Code inhärent - achte darauf, die Namensvergabe für Klassen, Variablen und Methoden so zu gestalten, dass der Code für (Weiter-)Entwickler selbsterklärend wird.

1.2 Vorkenntnisse

Ruby als hauptsächliche Programmiersprache während des gesamten Praktikums verwendet. Ich kenn mich im Betriebssystem Linux aus und finde mich in dem Text Editor Vim zurecht.

1.3 Vorarbeit

1.3.1 Online Zeitmessung

Wir haben die Zeitmessung bereits implementiert. Diese funktioniert aber nur bei erfolgreicher Verbindung zum Server. In der Zeitmessung geht es darum wie lange man an einem Task arbeitet. Arbeitet man zum Beispiel an dem Interface Task, wird zum starten der Zeitmessung folgendes im Command-Line-Interface eingegeben: `start /rdpm/interface`

Somit wird jetzt die Zeit gemessen, bis ich die Zeitmessung stoppe mit diesem Befehl: `stop /rdpm/interface`

1.3.2 Befehle

Entwicklung der RDPM-Applikation während 2 Wochen vor dem Projektstart, mit Schwerpunkt auf den Online-Teil des Time-Measureings.

Wir haben das Interface erstellt in dem man die Befehle definiert die aufgerufen werden können. Wir arbeiten mit dem Ruby Readline Library, mit dem man die eingegebenen Werten, welche in der Shell eingegeben wurden, übergeben kann. Es gibt normale User wie auch einen root User. Der root User kann zusätzliche Befehle ausführen.

Folgende Befehle können alle ausführen:

Befehl	Was passiert oder wann braucht man ihn?
,bt' oder backtrace	Braucht man wenn ein Fehler auftritt. Es zeigt in welchem File der Fehler passierte und von wo er kam.
,c' oder create	Um einen neuen Task zu erzeugen.
,cd'	Verzeichnis wechseln.
,d' oder data	Die Eigenschaften eines Tasks werden angezeigt. Wie z.B. wer hat die Verantwortung oder wie ist die Beschreibung des Tasks.
,e' oder edit	Man kann einen Task editieren. Z.B. die Beschreibung anpassen.
,li' oder login	Man logt sich beim Server ein.
,ls' oder list	Es werden alle untergeordneten Tasks angezeigt.
,mv' oder move	Ein Task wird verschoben.
,nt' oder name_tree	Der Name-Tree wird angezeigt. Die Tasks mit ihren untergeordneten Tasks.
,pwd'	Der Pfad in dem man sich momentan befindet wird angezeigt.
,go' oder start	Man startet das Zeitmessen. Dies braucht man um zu messen wie lang man an einem Task arbeitet für die Abrechnung.
,rs' oder resync	Löscht den Name-Tree
,st' oder stop	Das Zeitmessen wird gestoppt.
users	Alle User werden angezeigt.
whoami	Es wird angezeigt mit welchem User man eingeloggt ist.
quit oder exit	Die Verbindung zum Server wird getrennt.

Der root User hat folgenden zusätzlichen Befehl:

Befehl	Was passiert?
,cu' oder create_user	Es wird ein User erstellt.

1.3.3 Authentifikation

Dass sich der Client mit dem Server verbinden kann, muss man zuerst einen DSA Key erstellen. Dies wird erzeugt in der Shell mit folgendem Befehl: *ssh-keygen -tdsa*. Dies speichert einen public und private Key in den */home/user/.ssh* Ordner. Den public Key trägt man auf dem Server zu dem

dazugehörigen User-Account ein. Somit kann der Server Daten mit dem public Key verschlüsseln. Nur mit dem richtigen privaten Key kann man diese Daten wieder entschlüsseln. Es wird also überprüft ob der User auf dem Server eingetragen und zugelassen ist. Wir haben für diese Authentifikation ein eigenes Library erstellt mit dem Namen RRBA

1.3.4 Config Files

Es wurde für das Arbeiten mit dem Config File ein Library mit dem Namen RCLCONF erstellt. Das File befindet sich auf dem Server im Ordner /home/user/.rdpm/conf oder /etc/rdpm das File heisst rclconf.yaml. In diesem File befinden sich z.B. die Datenbank-Url, der Datenbank-User, die Server-Url, der Pfad zum root DSA-Key oder der root Name.

1.4 Firmenstandarts

ywesee -- intellectual capital connected

Coding-Standards
Version 1.0, 15.02.2005.

1.4.1 Filesystem

ywesee orientiert sich bezüglich Filesystem am Ruby-Standard:

Projekt-name		
	/bin/executable-files	(executables können auch Ruby-Files sein)
	/ext/require-pfad/c-extension-files	
	/lib/require-pfad/library-files	(alle Ruby-Files enden mit '.rb')
	/test/test-files	(alle Test-Files beginnen mit 'test_')

Zusätzlich möglich:

Projekt-name		
	/src/code-files	(Applikations-Code, der nicht als Library verwendet werden kann)
	/src/subsystem-name/code-files	(bei grösseren Projekten)
	/test/'test_' + subsystem-name/test-files	(bei grösseren Projekten, Files heissen gleich wie Code-Files)
	/doc/	(Document-Root bei Webprojekten)
	resources	(statische Web-Resources, z.B. CSS-Files, Images etc.)
	/etc/konfigurations-files	

Begründung:

- Standard-Compliance macht es z.B. einfacher ein Installations-Script zu erstellen.

1.4.2 File-Namen

Grundsätzlich gilt:

- 1 File pro implementierter Klasse (mehrere Klassen pro File nur in Ausnahmefällen, z.B. für Subklassen mit geringer Behaviordifferenz)
- File.name = Class.name.downcase + .rb
- TestFile.name = 'test_' + File.name

Ausnahme:

Bei grösseren Projekten, bei denen der Code in Subsystem-Directories abgelegt ist, beginnen Subsystem-Test-Directories mit 'test_'. Dann gilt TestFile.name = File.name

Begründung:

- Ruby erzwingt keinerlei Beschränkungen für die Nomenklatur. Trotzdem ist es für das Verständnis des Code-Lesers sinnvoll, schon im Filesystem eine Verbindung zwischen Code und Tests zu sehen.
- Um die require-Pfade eindeutig zu halten, haben Test-Files oder Subsystem-Test-Directories das Präfix 'test_'

1.4.3 Code-Formatierung

- Klassen: CamelCase:
Bsp: `class CodeStandardPolice`
- Konstanten: UPPERCASE, underscores für Worttrennung::
Bsp: `DOCUMENT_AUTHOR = 'hwyss'`
- Methoden und Variablen: lowercase, underscores für Worttrennung
Bsp: `def example_method()`
Bsp: `my_var = 10`
- Konditions-Ausdrücke, Methoden-Argumente (Definition und Aufruf) immer in Klammern:
Bsp: `if(foo == bar)`
Bsp: `def my_method(arg1, arg2)`
Bsp: `my_object.my_method('foo', 'bar')`
- Abstand vor und nach Operators:
Bsp: `1 + 2`
Bsp: `rule_number = 15`
- Abstand nach Komma:
Bsp: `my_object.my_method('foo', 'bar', 'baz')`
- Block-Definitionen: Geschweifte Klammern (Abstand vorher), Abstand vor und nach der Block-Argument-Definition:
Bsp: `hash.each { |key, value| printf("%i => %s", key, value) }`
- Mehrzeilige Block-Definitionen: Einrückung um ein Tab, Block-Argumente auf der Ursprungszeile, abschliessende geschweifte Klammer auf neuer Zeile und auf ursprünglicher Höhe.
Bsp: `hash.each { |key, value|
 printf("%i => %s", key, value)
}`

- **HashTable-Literals:** Wie Block-Definitionen, bei mehrzeiligen Hash-Literals Pfeil-Syntax und Werte horizontal aligniert

```
Bsp: table = {  
      1234      => 'simple',  
      385636213 => 'a little more complicated',  
    }
```

- Lange Code-Zeilen werden umgebrochen: Immer Backslash am Ende der Zeile, auch wenn vom Syntax nicht gefordert. Folgezeilen um ein Tab eingerückt.
- Bei Konditions-Ausdrücken werden logische Operatoren auf die neue Zeile genommen.

```
Bsp: @a_long_variable_name.and_a_very_long_method_name(with, many, \  
      arguments, to, boot)
```

```
Bsp: if(my_condition_variable == MY_CONDITION_CONSTANT \  
      && my_condition_method(my_condition_variable))
```

Begründung:

- Lesbarkeit in erster Linie eine Frage der Konsistenz und erst in zweiter Linie einer Frage der Formatierung. Die Regeln sind so gewählt, dass das Auge genügend Whitespace zur Orientierung erhält und der Editor keine zusätzlichen Zeilenumbrüche einfügen muss.

1.4.4 Namensgebung

- Getter-Methoden heissen gleich wie die Instanzvariable auf die sie sich beziehen:

```
Bsp: class Foo  
      def initialize  
        @bar = 23  
      end  
      def bar  
        @bar  
      end  
    end
```

(Dies ist ein Beispiel. In production code wird für ein reines Auslesen die 'attr_reader' Klassenmethode verwendet.)

- Setter-Methoden heissen gleich wie die Instanzvariable, auf die sie sich beziehen, mit dem Suffix '=':

```
Bsp: class Foo  
      def bar=(bar_value)  
        @bar = bar_value  
      end  
    end
```

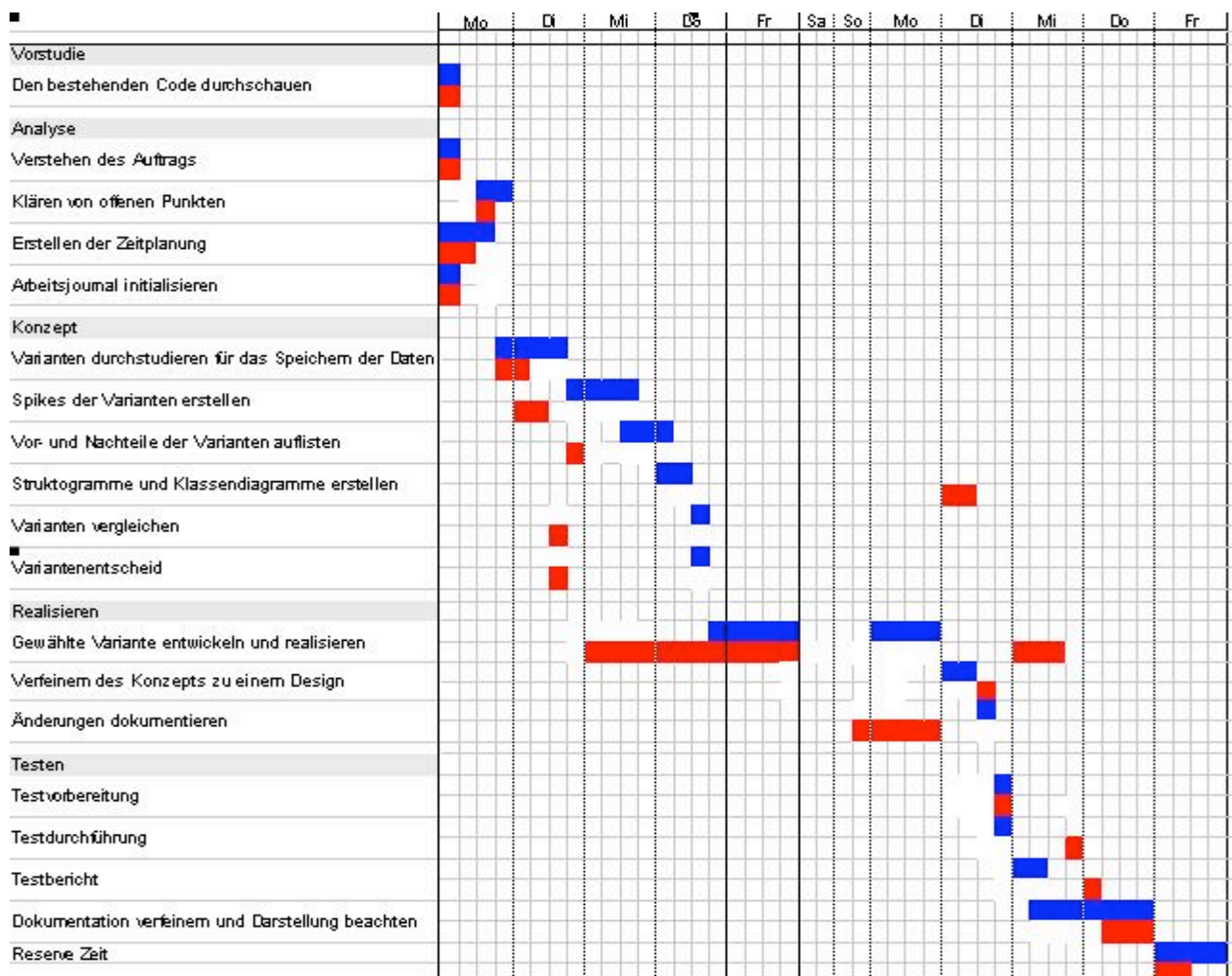
(Dies ist ein Beispiel. In production code wird für ein reines Zuweisen die 'attr_writer' Klassenmethode verwendet.)

- Keine Namenswiederholung. Eine Klasse 'Person' hat eine Getter-Methode 'name' und nicht 'person_name'.
- Descriptive Names: Namen von Klassen, Methoden und Variablen beschreiben die Funktion/Responsibility des Codes in Englisch.

1.4.5 Extreme-Programming die Wichtigsten Grundsätze

- Für jede neue Funktionalität wird ein Unit-Test (und gegebenenfalls ein Integration-Test) erstellt, bevor mit der Implementation begonnen wird.
- Do the simplest thing that could possibly work. (Die Antizipation von Kundenbedürfnissen durch den Entwickler und sogar durch den Kunden selbst führt meistens in die falsche Richtung).
- Refactor mercilessly: Code-Duplikation eliminieren, unpräzise Namensgebungen ändern.

1.5 Zeitplan



Soll
Ist

1.6 Arbeitsjournal

14. Februar 2005

Ausgeführte Arbeiten

Ich habe den Zeitplan erstellt. Danach mir Gedanken über das Speichern der Daten gemacht.

Erreichte Ziele

Zeitplan fertig erstellt. Somit habe ich einen geplanten Ablauf. Anhand dieses Planes werde ich arbeiten. Hab mir verschiedene Varianten für das Speichern der Daten überlegt und mir notiert.

Aufgetretene Probleme

Hatte Anfangsschwierigkeiten, konnte mir zuerst nicht vorstellen wie genau der Zeitplan aussehen muss.

Beanspruchte Hilfeleistung

n/a

Nacht- und Wochenendarbeiten

n/a

Vergleich mit dem Zeitplan

Liege in der Zeit.

15. Februar 2005

Ausgeführte Arbeiten

Ich habe einen Marshal, Yaml und Madeleine Spike erstellt. Danach mich für Yaml entschieden. Habe die ersten Tests programmiert, dass die Time-Measure Objekte im Yaml Format gespeichert werden.

Erreichte Ziele

Mich durch Spikes für das Speichern der Zeit Daten im Yaml-Format entschieden. Erste Tests bestehen.

Aufgetretene Probleme

War mir zuerst nicht sicher ob meine Überlegungen richtig sind.

Beanspruchte Hilfeleistung

Erzählte Hannes mein Vorgehen. Wollte wissen ob ich auf dem richtigen Weg bin.

Nacht- und Wochenendarbeiten

n/a

Vergleich mit dem Zeitplan

Liege in der Zeit.

16.Februar 2005

Ausgeführte Arbeiten

Heute ging es darum das Speichern der Time Measuring fertig zu programmieren. Ebenfalls muss in einem Thread immer wieder geschaut werden, ob man sich mit dem Server verbinden kann. Sobald man verbunden ist, werden aus dem Yaml File die Daten auf den Server geladen und dort die Zeit bei den Tasks hinzugefügt.

Erreichte Ziele

Hab die Funktionalität für das erstellen und senden des Yaml-Files fertig programmiert und dazu die Tests geschrieben. Mit dem Experten über die Facharbeit gesprochen und einen nächsten Termin vereinbart.

Aufgetretene Probleme

Hatte zuerst Mühe aussagekräftige Tests zu erstellen. Da es nicht sehr einfach ist Tests zu erstellen in dem ein Yaml-File erstellt und geschrieben wird. Ich musste noch schauen wo ich am besten das Synchronisieren mit dem Server ausführe.

Beanspruchte Hilfeleistung

Kurz meine Vorgehensweise am Hannes erklärt. Wurde als guten Weg genannt.

Nacht- und Wochenendarbeiten

Arbeitsjournal nachgeführt.

Vergleich mit dem Zeitplan

Habe das Vergleichen der Tasks noch nicht dokumentiert dafür bin ich beim Realisieren besser in der Zeit.

17.Februar 2005

Ausgeführte Arbeiten

Habe ein paar Test geschrieben für den Ablauf der Tasks. Leider funktionieren noch nicht alle. Diese Arbeit war ziemlich nervenaufreibend. Man muss sich genau überlegen was als nächstes passieren muss. Ich bin fast vor der Vollendung. Ebenfalls beachtete ich stets die Variablebezeichnungen.

Erreichte Ziele

Habe neue Tests erstellt. Und angefangen die Tests zum laufen bringen.

Aufgetretene Probleme

Später am Abend konnte ich mich nicht mehr konzentrieren, da ich zu müde geworden bin. Bei dieser Arbeit bei der man den Ablauf richtig verstehen muss, war es wichtig sich konzentrieren zu können.

Beanspruchte Hilfeleistung

Mike hat mir bei einer Variabelbezeichnung geholfen.

Nacht- und Wochenendarbeiten

Arbeitsjournal nachgeführt

Vergleich mit dem Zeitplan

Liege beim Realisieren vorne, aber bei der Dokumentation hinke ich hintennach.

18.Februar 2005

Ausgeführte Arbeiten

Habe die Tests überarbeitet. Die Methode *stored?* gab nicht den richtigen Rückgabewert zurück. Sie müsste *true* oder *false* zurückgeben, dies habe ich dann auch so implementiert. Danach habe ich mich weiter mit dem Problem des Ablaufes der Tasks beschäftigt. Ich habe die Funktion welche das automatische Starten des letzten Taskes im *suspend_tasks* Array ausführt, fertig gestellt. Habe dies dann in der Shell getestet.

Erreichte Ziele

Man kann jetzt mehrere Tasks starten. Startet man den einen zweiten Taskes wird der erste automatisch gestoppt aber in einen *suspend_tasks* Array geschrieben, da man ihn ja noch nicht von Hand gestoppt hat. Hier ein Beispiel:

Befehl den man eingibt	Was passiert?
Start foo	Foo wird in den <i>suspend_tasks</i> Array geschrieben
Start bar	Bar wird in den <i>suspend_tasks</i> Array geschrieben. Ebenfalls wird Task-Foo gestoppt.
Stop	Task-Bar wird gestoppt und der Task-Foo welcher sich noch im <i>suspend_tasks</i> Array befindet automatisch wieder gestartet.

Aufgetretene Probleme

Hatte zuerst Probleme da es ein automatisches Stopp gibt beim Starten eines weiteren Tasks und bei dem darf nichts mit dem *suspend_tasks* Array passieren. Ebenfalls gibt es ein automatisches Starten nach dem Stoppen eines Tasks. Darum musste ich eine *internal_stop* und *internal_start* Methode erstellen.

Beanspruchte Hilfeleistung

n/a

Nacht- und Wochenendarbeiten

Arbeitsjournal nachgeführt.

Vergleich mit dem Zeitplan

Bin beim Realisieren besser in der Zeit als geplant, aber mit dem Dokumentieren ein wenig in Verzug.

21. Februar 2005

Ausgeführte Arbeiten

Habe den ganzen Tag in die Dokumentation investiert. Ich habe das Kapitel über die Spikes fertig erstellt, ebenfalls die Aufgabenstellung und Erklärung. Habe ein Diagramm erstellt vom ganzen RDPM.

Erreichte Ziele

Habe 13 Seiten an der Dokumentation geschrieben.

Aufgetretene Probleme

Habe gemerkt, dass es ziemlich viel Zeit braucht richtige Sätze zu finden die gut Verständlich sind. Das RDP Diagramm war zuerst nicht sehr einfach.

Beanspruchte Hilfeleistung

n/a

Nacht- und Wochenendarbeiten

Habe am Sonntag 90 Minuten an der Dokumentation gearbeitet.

Vergleich mit dem Zeitplan

Habe ein paar Tasks, die ich eigentlich früher machen wollte erst jetzt erledigt. Dafür bin ich im Realisieren voraus.

22. Februar 2005

Ausgeführte Arbeiten

Ich habe mich heute weiter mit der Dokumentation beschäftigt. Ich habe den Entscheid für das Speichern der Daten genauer erläutert. Ebenfalls habe ich begonnen das Kapitel Realisieren zu dokumentieren.

Erreichte Ziele

Habe mir gut überlegt, was im Kapitel Realisieren beschrieben werden muss.

Aufgetretene Probleme

Welche Themen müssen beschrieben werden.

Beanspruchte Hilfeleistung

n/a

Nacht- und Wochenendarbeiten

Arbeitsjournal nachgeführt.

Vergleich mit dem Zeitplan

Ich liege im Zeitplan.

23. Februar 2005

Ausgeführte Arbeiten

Ich habe heute meine Aufgabenstellung nochmals genau durchgelesen. Danach habe ich weitere Funktionen implementiert und Tests dazu geschrieben. Man kann jetzt *stop_all* aufrufen und alle Tasks werden gestoppt. Bei jedem Starten oder Stoppen eines Tasks werden Statusmeldungen ausgegeben.

Erreichte Ziele

Ich habe alle Aufgaben erfüllt. Die Statusmeldungen werden richtig ausgegeben. Der Ablauf mit dem Starten und Stoppen der Tasks funktioniert ebenfalls richtig.

Aufgetretene Probleme

War mal wieder eine sehr nerven aufreibende Arbeit. Bei der Statusmeldung war es nicht ganz einfach den richtigen Pfad auszugeben.

Beanspruchte Hilfeleistung

n/a

Nacht- und Wochenendarbeiten

Arbeitsjournal nachgeführt und ein wenig an der Dokumentation geschrieben.

Vergleich mit dem Zeitplan

Habe ein wenig länger als geplant für das realisieren gebraucht. Dafür war ich schneller bei der Variantenentscheid.

24. Februar 2005

Ausgeführte Arbeiten

Habe heute die Tests angepasst. Die Dokumentation wurde erweitert mit den Kapiteln Installation und Benutzeranleitung

Erreichte Ziele

Dokumentation erweitert. Die Arbeit wurde von mehreren Leuten durchgelesen und korrigiert.

Aufgetretene Probleme

n/a

Beanspruchte Hilfeleistung

Meine Mutter und Mike haben die Arbeit durchgelesen.

Nacht- und Wochenendarbeiten

An der Dokumentation gearbeitet und das Arbeitsjournal nachgeführt.

Vergleich mit dem Zeitplan

Werde mit der Arbeit fertig. Habe aber ein wenig der Reserve Zeit gebraucht.

25. Februar 2005

Ausgeführte Arbeiten

Dokumentation verfeinert.

Erreichte Ziele

Dokumentation fertiggestellt.

Aufgetretene Probleme

n/a

Beanspruchte Hilfeleistung

n/a

Nacht- und Wochenendarbeiten

n/a

Vergleich mit dem Zeitplan

Stimmt gut überein.

2 Project

2.1 Verfeinerung des Auftrags

In meinem Auftrag geht es hauptsächlich um zwei Aufgaben.

- Das Offline Arbeiten mit der Zeitmessung. Dies bedeutet man muss die Daten zwischenspeichern. Besteht später eine Verbindung zum Server, wird der Server mit den abgespeicherten Daten des Clients auf den neusten Stand gebracht.
- Das Zeitmessen von zwei oder mehreren Project/Tasks
Ich muss also die Task welche gestartet wurden zwischenspeichern z.B. in einem Array.

2.2 Planung

Mein Projekt wird nach dem Extreme-Programming Ansatz durchgeführt. Bei dieser Vorgehensweise schreibt man von Anfang an Code. Die Dokumentation wird auf ein Minimum reduziert. Da Extreme-Programming meistens mit Test-First kombiniert wird, ist ein Grossteil der Dokumentation eines Systems in den Unittests enthalten.

2.3 Einführung in das RDPM

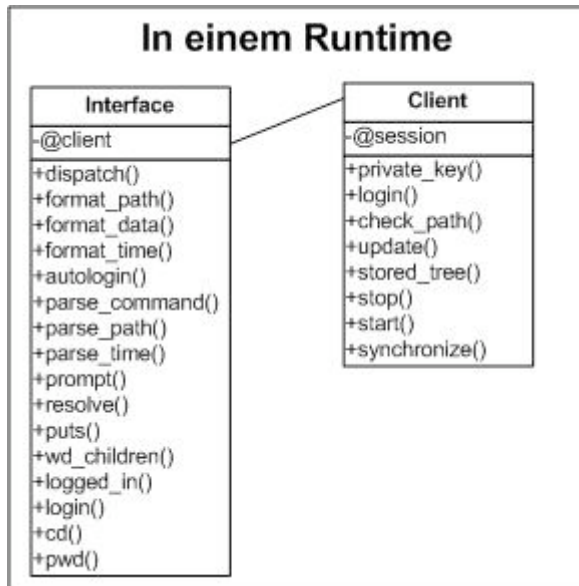
Da mein Projekt eine Funktion des RDPM ist, stelle ich hier kurz das RDPM Library vor.

2.3.1 Was ist RDPM?

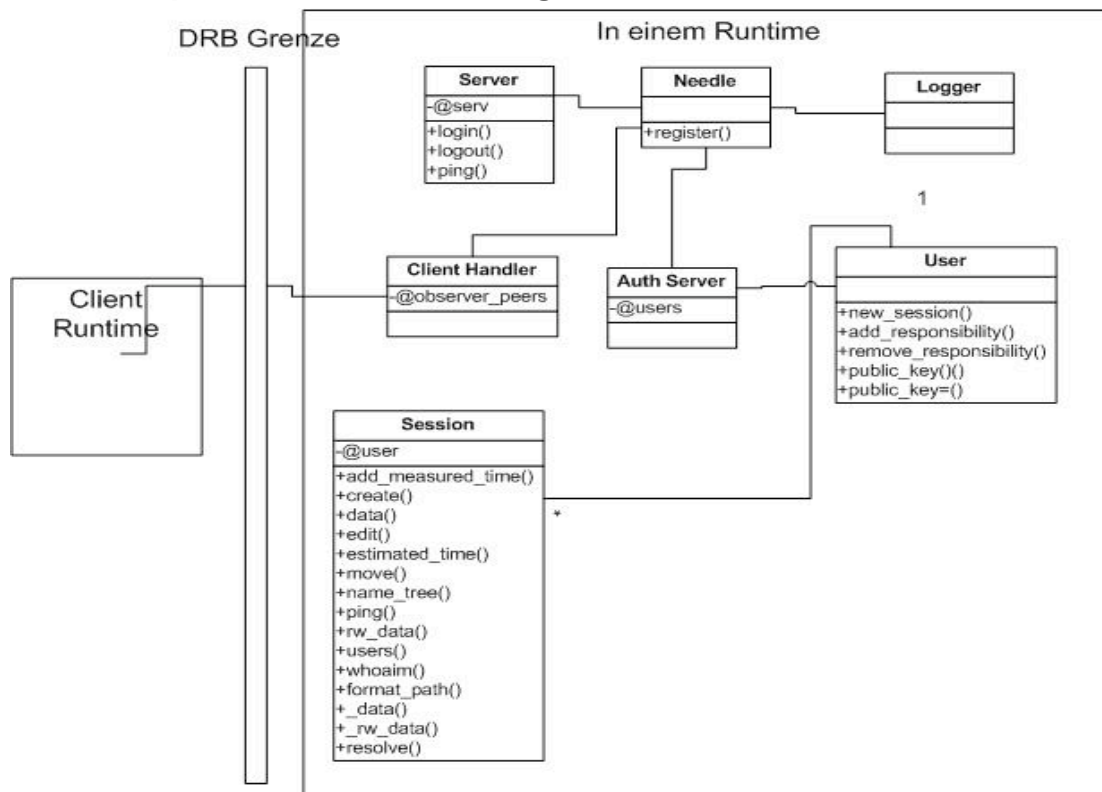
RDPM steht für Ruby Distributed Project Manager. Es handelt sich um ein Projektmanagement Tool, welches man mit der Shell benützt. Es besteht aus einem Client und einem Server.

Wir brauchen dieses Tool für die Übersicht unsere Projekte und die Abrechnung am Ende eines Projektes. Es wird gemessen, wie lang und wer in einem Projekt bzw. Task gearbeitet hat.

2.3.2 Simplifiziertes Klassendiagramm des Clients



2.3.3 Simplifiziertes Klassendiagramm des Servers



2.4 Konzept für das Abspeichern der Time-Measuring Daten

Wie speichere ich die Time-Measuring Daten, welche beim Offline Arbeiten erzeugt werden, um sie später an den Server zu senden?

Ich kam zu folgenden möglichen Varianten:

- Madeleine
- Mnemonic
- XML
- YAML

- Marshal
- Odba
- Relationale DB

Ich habe für die Varianten Madeleine, Marshal und YAML ein Spike erstellt, um genauer herauszufinden, welche die zutreffendste Speicherart ist.

2.4.1 Madeleine Variante

Madeleine ist ein Ruby Library mit dem man Daten anhand eines Snapshot speichern kann, welche in einem File abgelegt werden.

Ein kleiner Spike:

```
require 'madeleine'
require 'madeleine/automatic'

class Command
  def execute(system)
    system.count
  end
end

class Client
  include Madeleine::Automatic::Interceptor
  def initialize
    @example = 0
  end
  def count
    @example += 1
    puts @example
  end
end

end
STORAGE_PATH = File.expand_path("~/bitkeeper/rdpm/spike/")
madeleine = AutomaticSnapshotMadeleine.new(STORAGE_PATH) {
  Client.new()
}
madeleine.execute_command(Command.new)
madeleine.take_snapshot
```

Output beim ersten Mal laufen lassen:

15

Output beim zweiten Mal laufen lassen:

16

Es ist ersichtlich, dass sich die Variable *@example* bei jedem Speichern um 1 erhöht. Somit ist bewiesen, dass die *@example* Variable im Snapshot gespeichert wird und nicht immer wieder auf den Wert 0 gesetzt wird.

2.4.2 Marshal Variante

Marshal ist ein Ruby-Modul um Daten zu Dumpen und Laden.
Ein kleiner Spike:

```
class TimeMeasure
  def initialize(count)
    @count = count
  end
  def count_up
    @count += 5
  end
end
class Client
  attr_reader :count
  def initialize
    @foo = "foo"
    @bar = 123
    @count = 0
  end
  def time_measure
    @count = TimeMeasure.new(4).count_up
  end
end
client = Client.new
puts "Count before dump"
puts client.count
dump = Marshal.dump(client)
puts "Dump Inspect"
puts dump.inspect
client = Marshal.load(dump)
puts "Client Inspect"
puts client.inspect
client.time_measure
puts "New Count"
puts client.count
```

Der Output sieht wie folgt aus:

```
Count before dump
0
Dump Inspect
"\004\010o:\vClient\010:\t@bari\001{:\v@counti\000:\t@foo\\"\010foo"
Client Inspect
#<Client:0xb7d87160 @bar=123, @count=0, @foo="foo">
New Count
9
```

In diesem Spike habe ich zuerst die Klasse Client mit ihren Variablen *@foo*, *@bar* und *@count* gedumpt. Dann wird der Dump wieder geladen und die Methode *time_measure* auf den geladenen Client angewendet. In der Methode wird der Wert 4 an der Klasse TimeMeasure übergeben. Der Wert wird in der *initialize* Methode der *@count* Variable zugeordnet. Danach in der TimeMeasure Klasse mit der Methode *count_up* um 5 erhöht. Somit hat die Variable *@count* den neuen Wert 9.

2.4.3 YAML Variante

Yaml ist ein Daten Format wie Xml, mit dem man Daten in einem File abspeichern kann. Yaml ist bei der Programmiersprach Ruby integriert.

Ein kleiner Spike:

```
require 'yaml'

class Client
  def initialize
    @foo = "foo"
    @count = 123
  end
  def count_up
    @count += 2
  end
end

client = Client.new
yaml_obj = YAML::dump(client)
puts yaml_obj
ruby_obj = YAML::load(yaml_obj)
puts „Ist die Client Klasse nach dem dumpen und loaden noch die gleiche?“
puts client.class == ruby_obj.class
ruby_obj.count_up
yaml_obj = YAML::dump(ruby_obj)
puts yaml_ob
```

Der Output sieht wie folgt aus:

```
--- !ruby/object:Client
count: 123
foo: foo
Ist die erstellte Klasse nach dem dumpen und loaden noch die gleiche?
true
--- !ruby/object:Client
count: 125
foo: foo
```

Dieser Spike ist ähnlich aufgebaut wie der Spike des Marshals.

Es wird zuerst ein Client im YAML Format gedumpt mit den Variabeln *@foo* und *@count*. Die Variable *@count* hat zu Beginn den Wert 123. Danach wird der *yaml_dump* wieder geladen. Das erste Objekt wird mit dem Neuen verglichen ob es noch dasselbe ist, nach dem Dumpen und Laden. Auf das erneut geladene Objekt wird die Methode *count_up* ausgeführt. Die *@count* Variable steigt somit um 2. Dumpt man das Objekt wieder, sieht man das die Variable *@count* einen anderen Wert hat als zuvor.

2.4.4 Variantenvergleich

Ich verglich die Vorteile wie auch die Nachteile der verschiedenen Varianten.

Variante	Vorteile	Nachteile
Madeleine	- Es gibt eine Methode in der man automatisch den Snapshot updaten kann. - Ist ein Ruby Library.	Kenne mich zu wenig aus. Würde mich zu viel Zeit kosten dieses Library perfekt zu verstehen.
Mnemonic	Da ich dieses Library gar nicht kenne, kann ich keine Vorteile auflisten	Kenne diese Library gar nicht. Ich weis, es ist sehr ähnlich wie Madeleine

XML	• Sehr bekanntes Daten Format.	• Finde das XML- Format nicht sehr übersichtlich gegenüber dem YAML-Format. • Kenne mich nicht sehr gut aus, habe es bisher einmal kurz gebraucht.
YAML	• Ruby hat das YAML Library standardmässig dabei. • Gibt <code>to_yaml</code> Methode.	• Ist im Allgemeinen nicht so gekannt.
Marshal	• Ist ein Ruby Modul. Man muss nichts installieren. • Sehr einfach zu implementieren.	• Ich muss diesen Dump noch in ein File abspeichern. Da kann ich gerade so gut YAML oder XML verwenden.
ODBA	• Sehr einfach die Daten abzuspeichern mit <code>odba_store</code>.	• Man muss eine Datenbank auf dem Client installieren.
Relationale DB	• Tabellen sind übersichtlich	• Man muss eine Datenbank auf dem Client installieren • Für so wenig Daten zu aufwendig • Bei jeder Änderung an der DB muss man alle Clients anpassen.

2.4.5 Entscheid

Für mich war klar, dass die ODBA und Relationale DB Variante nicht in Frage kam, da es zu umständlich wäre, für jeden Benutzer des RDPM eine Datenbank aufzusetzen. Falls an der Struktur des Time-Measuring was geändert wird, könnte es sein, dass man alle Datenbanken anpassen müsste, was auch gegen diese zwei Varianten spricht.

Ich entschied mich ebenfalls gegen die Varianten Mnemonic und Madeleine, welches zwei Ruby Libraries sind, weil ich diese Libraries noch nie verwendet habe und somit der Aufwand um diese Varianten kennenzulernen zu gross ist. Für grössere Datenmengen wären diese Varianten sicher sehr geeignet.

Es blieben nur noch die Varianten YAML, XML oder Marshal. Die Marshal Variante fiel für mich weg, da ich den Dump welcher erstellt wird ebenfalls in ein File speichern muss, also kann ich gerade so gut die Daten in ein YAML oder XML File speichern.

Also blieben mir noch die YAML und XML Varianten. XML ist sicher das bekanntere und mehr gebrauchte Format, jedoch habe ich es bisher nur kurz kennengelernt. Das YAML-Format ist nicht sehr verbreitet, aber bei den Ruby Programmierern sehr beliebt, weil Ruby das YAML Library standardmässig integriert hat. Ebenfalls ein wichtiger Punkt ist die Darstellung des File Formats. Das YAML- Format finde ich gegenüber dem XML - Format viel verständlicher.

Hier die verschiedenen Darstellungen:

XML:	YAML:
<pre><xml> <list> <item>text</item> <item><hash> <key>key</key> <val>value</val> </hash></item> ... </list> </xml></pre>	<pre>--- - text - key: value k2: v2</pre>

Ich entschied mich darum für das Speichern der Daten im YAML-Format.

2.5 Realisierung

Es ging hauptsächlich um zwei Tasks welche realisiert werden mussten.

- Das Abspeichern der Time-Measuring Daten im YAML-Format
- Zeitmessung für parallele Arbeiten

In den Unterkapiteln werden diese Tasks genauer erläutert und erklärt auf welche Probleme ich achten musste und wie ich sie realisiert habe.

2.5.1 Vorgehensweise

Ich überlegte mir welche Methoden es geben wird und welche Funktion sie hat. Zuerst wird ein Test erstellt für die Methode welche ich programmieren möchte. Die ganze Arbeit befindet sich eigentlich schon im Test, da man sich dort überlegen muss, welche Rückgabewerte die einzelnen Aufrufe ergeben und welche Funktionen in der Methode, welche wir testen, aufgerufen werden. Man nennt diese Vorgehensweise Test-First. Diese Arbeitsweise kommt von der Extreme-Programming Philosophie.

2.5.2 Das Abspeichern der Time-Measuring Daten im YAML-Format

Wie im Konzept erwähnt wurde, habe ich mich für das Speichern im YAML-Format entschieden.

Die erste Frage die ich mir stellen musste bevor ich diese Aufgabe realisiere, war was muss ich eigentlich Speichern bzw. was muss ich später dem Server übergeben?

Die Server Methode *add_measured_time()* erwartet zwei Argumente. Das Time-Measure Objekt wie auch den Pfad zum Task. Diese zwei Werte muss ich zwischenspeichern bevor sie in ein YAML geschrieben werden. Dies machte ich mit einem Array den ich Queue nenne.

Hier ein kleiner Code Ausschnitt:

```
connection {
  @session.add_measured_time(@current_path, time_measure)
}
rescue ConnectionError
  @queue.push([@current_path, time_measure])
  file_path = create_file_path('queue.yaml')
  File.exist?(file_path) ? File.delete(file_path) : nil
  File.open(file_path, 'w') { |fh|
    fh.puts @queue.to_yaml
  }
end
```

In der *connection{}* Methode wird überprüft ob eine Verbindung zum Server besteht. Ist das nicht der Fall wird ein Connection-Error geworfen und der Pfad wie auch das Time-Measure Objekt in den Queue Array gespeichert. Danach wird der Queue Array in das Yaml-File "queue.yaml" geschrieben. Existiert das File schon wird es zuerst gelöscht und dann neu erstellt.

2.5.3 Das Synchronisieren mit dem Server

In der Interface-Klasse gibt es einen Thread der anhand eines interval Wertes welcher in dem Config-File definiert ist, schaut ob eine Verbindung zum Server besteht.

Hier ein Code Ausschnitt:

```
@connector_thread = Thread.new {
  Thread.current.abort_on_exception = true
  loop {
    _autologin(@autologin_server)
    sleep(@config.al_interval.to_f)
  }
}
```

Im Thread läuft also stets ein Loop.

Im Loop wird die Methode *_autologin()* aufgerufen.

Diese Methode sieht so aus:

```
def _autologin(server)
  unless(logged_in?)
    login(server)
    @client.synchronize
  end
  rescue DRb::DRbConnError, RRBA::AuthenticationError
end
```

Es wird überprüft, ob man schon eingeloggt ist, falls dies nicht so ist, wird versucht sich einzuloggen mit der Methode *login()*. Wirft dies keinen Fehler, wird die *synchronize()* Methode vom Client aufgerufen.

In der `synchronize()` Methode wird das Yaml-File geöffnet und das Objekt in die Variable `Queue` geschrieben. Danach werden die Werte des `Queue Arrays` per `add_measure_time()` Methode von der Session an den Server übergeben. Dort wird die Time-Measure Zeit den Tasks zuordnen. Nach dem Synchronisieren setze ich den `Queue Array` wieder leer.

Es gibt eine `@config.al_interval` Variable, falls die `_autologin()` Methode einen Error wirft, schläft der Thread solange wie der Wert der Variable ist. Also hat die Variable `@config.al_interval` den Wert 60, schläft der Loop 60 Sekunden lang.

2.5.4 Zeitmessung für die parallele Arbeit an zwei oder mehr Projekten/Tasks

In dieser Arbeit geht es um das parallele Zeitmessen von mehreren Tasks.

Hier ein Beispiel das in dem Interface eingegeben werden könnte:

```
start /rdpm          -> Das Zeitmessen für den Rdpm-Task wird gestartet.
start /rdpm/interface -> Das Zeitmessen für den Interface-Task wird gestartet und
                        der Task Rdpm automatisch gestoppt.
stop                 -> Das Zeitmessen für den Interface- Task wird gestoppt
                        und der Rdpm automatisch wieder gestartet.
stop                 -> Der Rdpm-Task wird gestoppt. Somit sind alle gestoppt.
start                -> Das Zeitmessen für den Rdpm-Task wird wieder
                        gestartet. Immer der Pfad bei der letzten Zeitmessung wird
                        abgespeichert .
stop                 -> Wieder alle Zeitmessungen sind gestoppt.
```

Ich löste diese Aufgabe mit fünf Methoden in der Client Klasse. Ich brauchte für das stoppen und starten je 2 Methoden da es ein automatisches stoppen wie starten gibt. Bei denen der `suspend_tasks` Array nicht verändert werden darf. Der `suspend_tasks` Array besteht aus den Task welche gestartet sind, aber noch nicht manuell per Befehl gestoppt wurden. Dann gibt es noch eine `stop_all`. Bei dem der `suspend_tasks` Array leer gesetzt wird und der letzte Pfad als current Pfad gespeichert wird.

Bei jedem Befehl Aufruf im Interface wird eine Statusmeldung ausgegeben. Es gibt bisher folgende Statusmeldungen.

- Status: Working on /rdpm
- Status: Pausing on /rdpm
- Status: stopped

2.6 Testen

Wie schon mehrmals in dieser Dokumentation erwähnt arbeiten wir nach der Extrem- Programming Philosophie. Dies bedeutet für das Testen, dass zuerst Test geschrieben werden dann der Code. Dies hat den Vorteil das man sich durch den Test Gedanken macht was die Funktion können muss. Und erweitert man zu einem späteren Zeitpunkt eine Methode, kann man durch den Test überprüfen ob die andere Funktionalität noch besteht

Ich habe aber hier mit dem Blackboxverfahren Tests erstellt, da es Sinnvoll ist zu kontrollieren ob die Rückgabewerte bei den Zeitmessungen von mehreren Tasks korrekt sind und sich die Time-Measure Daten richtig verändert haben.

2.6.1 Testprotokoll

Testfall1

Ablauf	Was erwarte ich?	Resultat
Data /rdpm	measured_time = 0	measured_time = 0
Data /rdpm/interface	measured_time = 3	measured_time = 3
Start /rdpm	working on /rdpm	working on /rdpm
5 Minuten später. Start /rdpm/interface	working on /rdpm/interface	working on /rdpm/interface
3 Minuten später stop	working on /rdpm	working on /rdpm
2 Minuten später stop	pausing on /rdpm	pausing on /rdpm
Data /rdpm	measured_time = 7	measured_time = 7
Data /rdpm/interface	measured_time = 6	measured_time = 6

Testfall2

Ablauf	Was erwarte ich?	Resultat
Data /rdpm	measured_time = 0	measured_time = 0
Data /rdpm/interface	measured_time = 3	measured_time = 3
Start /rdpm	working on /rdpm	working on /rdpm
5 Minuten später. Start /rdpm/interface	working on /rdpm/interface	working on /rdpm/interface
3 Minuten später stop	working on /rdpm	working on /rdpm
2 Minuten später stop	pausing on /rdpm	pausing on /rdpm
start	working on /rdpm	working on /rdpm
10 Minuten später start /rdpm/interface	working on /rdpm/interface	working on /rdpm/interface
3 Minuten später stop	working on /rdpm	working on /rdpm
2 Minuten später stop	pausing on /rdpm	pausing on /rdpm
Data /rdpm/interface	measured_time = 9	measured_time = 9
Data /rdpm	Measured_time = 19	Measured_time = 19

Testfall3

Ablauf	Was erwarte ich	Resultat
Data /rdpm	measured_time = 0	measured_time = 0
Data /rdpm/interface	measured_time = 3	measured_time = 3
Start /rdpm	working on /rdpm	working on /rdpm
5 Minuten später. Start /rdpm/interface	working on /rdpm/interface	working on /rdpm/interface
3 Minuten später stop all	pausing on /rdpm/interface	pausing on /rdpm/interface
Data /rdpm	measured_time = 5	measured_time = 5
Data /rdpm/interface	measured_time = 6	measured_time = 6

2.6.2 Testbericht

Meine Blackboxtests haben alle erfolgreich bestanden. Somit ist die Funktionalität des Programms gewährleistet. Die Test Suite besteht ebenfalls, somit wurden keine anderen Funktionen beschädigt.

2.7 Installationsanleitung

Um die folgenden Installationen durchzuführen, müssen Sie folgende Libraries installiert haben.

- Mod-Ruby
- Ruby-Dbi
- Ruby
- Needle kann nicht per emerge Tool installiert werden sondern mit diesen Befehlen: *sudo emerge rubygems, sudo gem install needle*
- RCLConfig(Bei uns erhältlich)
- RRBA (Bei uns erhältlich)
- Odba (Bei uns erhältlich)

2.7.1 Client-Installation

Um das RDPM Library zu installieren, braucht man zuerst den Source Code welcher bei uns gratis erhältlich ist. Ist der Source Code vorhanden, muss man in das RDPM Verzeichnis wechseln. Es müssen dort folgende Befehle in der Shell ausgeführt werden:

```
sudo ruby install.rb config  
sudo ruby install.rb setup  
ruby install.rb install
```

Man sollte dann einen Ordner mit dem Namen ".rdpm" im /home/user/ Verzeichnis erstellen und dort ein File rclconfig.yaml. In diesem File wird z.B. die Server Url definiert:

```
al_url:druby://localhost:10000
```

Es muss noch ein DSA-Key erstellt werden. Dies macht man mit diesem Befehl
ssh-keygen -tdsa

2.7.2 Server-Installation

Postgres und Ruby-Postgres muss installiert werden. Dies kann man auf einem Computer mit der Gentoo Distribution mit emerge postgres ruby-postgres installieren.

Für die Server-Installation müssen die Schritte des Client ebenfalls ausgeführt werden. Im rclconfig.yaml muss zusätzlich der Pfad zum Root DSA-Key gesetzt werden.

Erstellen der Datenbank auf dem Server

Es muss eine RDPM-Datenbank erstellt werden. Dafür muss man zuerst in das Admin Tool des Postgres.

```
Psql -U postgres template1
```

Dann wird die RDPM Datenbank erstellt.

```
Create Database rdpm;
```

Mit \q loggt man sicher aus.

Das man mit der DB arbeiten kann, braucht man noch das ODBA Framework welches man auch bei uns erhalten kann.

Danach geht man in die neu erstellte DB:

```
psql -U postgres rdpm
```

Es müssen 3 Tabellen und eine Funktion erstellt werden.

Erstellen der *object* Tabelle:

```
CREATE TABLE object (  
    odba_id integer NOT NULL,  
    content text,  
    name text,  
    prefetchable boolean,  
    PRIMARY KEY(odba_id),  
    UNIQUE(name)  
);
```

Erstellen der *object_connection* Tabelle:

```
CREATE TABLE object_connection (  
    origin_id integer,  
    target_id integer,
```

```
PRIMARY KEY(origin_id, target_id)
);
CREATE INDEX target_id_index ON object_connection (target_id);
CREATE INDEX origin_id_index ON object_connection (origin_id);
```

Erstellen der *collection* Tabelle:

```
CREATE TABLE collection (
    odba_id integer NOT NULL,
    key text,
    value text,
    PRIMARY KEY(odba_id, key)
);
```

Die *update_object* Funktion:

```
CREATE OR REPLACE FUNCTION update_object(integer, text, text, boolean)
RETURNS BOOLEAN AS '
DECLARE
    v_old_name text;
    v_new_name text;
BEGIN
    SELECT INTO v_old_name name FROM object WHERE odba_id = $1;
    IF FOUND THEN
        IF $3 IS NULL THEN
            v_new_name := v_old_name;
        ELSE
            v_new_name := $3;
        END IF;
        UPDATE object
        SET content = $2, name = v_new_name, prefetchable = $4
        WHERE odba_id = $1;
    ELSE
        INSERT INTO object (odba_id, content, name, prefetchable)
        VALUES ($1, $2, $3, $4);
    END IF;
    RETURN FOUND;
END;
' LANGUAGE plpgsql;
```

2.8 Benutzeranleitung

Die Server-Applikation wird mit *rdpm/bin/rdpmd* gestartet und die Client-Applikation mit */rdpm/bin/rdpm* gestartet.

Besteht eine Verbindung zum Server, erscheint in der Konsole folgendes.

```
rdpm:username>
```

Danach kann man z.B. den Befehl *start /rdpm* eingeben (Der Rdpd-Task muss existieren). Welche Befehle vorhanden sind, wurde im Kapitel "Vorarbeiten" schon erwähnt.

2.9 Schlusswort

Ich freue mich dass meine Facharbeit bei Ywesee Verwendung findet. Im Grossen und Ganzen lief die Arbeit ziemlich gut. Es gab sicher Höhen und Tiefen, aber das kommt bei jeder Arbeit vor. Manchmal muss man das Problem bei Seite legen und sich eine Pause gönnen. Wieder mit frischem Kopf an der Arbeit, geht es meistens besser vorwärts.

3 Anhang

3.1 Literaturverzeichnis

Internetseiten

www.ruby-doc.com

Buch

Programmieren mit Ruby



3.2 Glossar

Authentifikation

Bei der Authentifikation wird der Anwender eines Systems auf seine tatsächliche Identität untersucht. So kann verhindert werden, dass ungerechtfertigt Dienste und Systemressourcen in Anspruch genommen werden können.

Command Line Interface

Eine Schnittstelle, über die Befehle direkt eingegeben werden können.

Viele Systeme verfügen sowohl über eine solche Schnittstelle für den erfahrenen Benutzer als auch über eine menügesteuerte Oberfläche, mit der einfacher gearbeitet werden kann. Dafür ist das Command-Line-Interface in der Regel die direkteste Art, um bestimmte Funktionen zu nutzen.

DB

Datenbank

Emerge Tool

Gentoo Package Manager für das installieren und deinstallieren von Programmen.

Task

Die Aufgabe

ODBA

Framework, welches Objekte persistent abspeichert.

Test Suite

Mit der Testsuite werden alle Tests laufen gelassen, welche man programmiert hat.

RCLCONF

A Ruby Configuration Library

RRBA

Ruby Role Based Authentication

Shell

Ein Kommandozeileninterpreter, auch *Shell* genannt, ist eine Software, welche eine Zeile Text in der Kommandozeile einliest, diesen Text als Kommando interpretiert und ausführt.

Spike

Eine sehr kleine Applikation, die in der Regel erstellt wird, um mit einer Technologie vertraut zu werden oder die Effizienz von verschiedenen Lösungsansätzen zu vergleichen. Spikes werden normalerweise nach Gebrauch gelöscht, sollen aber im vorliegenden Prüfungsfall zu Dokumentationszwecken behalten werden.

3.3 Code Anhang

Alles was fett geschrieben ist, habe ich programmiert.

File: test_client.rb

```
def test_start_with_suspend_tasks
  @config.__next(:rdpm_dir) { @datadir }
  @client.instance_variable_set('@session', @session)
  @session.__next(:name_tree) { @tree }
  @session.__next(:add_measured_time) { |pth, measure| }
  assert_nil(@client.current_path)
  assert_nil(@client.start_time)
  path = ['Project1', 'Task2']
  path2 = ['Project1']
  @client.start(path)
  assert_equal(path, @client.current_path)
  assert_instance_of(Time, @client.start_time)
  assert_equal([path], @client.suspend_tasks)
  @client.start(path2)
  assert_equal(path2, @client.current_path)
```

```
    assert_instance_of(Time, @client.start_time)
    assert_equal([path, path2], @client.suspend_tasks)
end
def test_stop_with_suspend_tasks_no_path
  assert_raises(RuntimeError) { @client.stop }
  path = ['Project1', 'Task2']
  path2 = ['Project1']
  @config.__next(:rdpm_dir) { @datadir }
  @client.instance_variable_set('@suspend_tasks', [path, path2])
  @client.instance_variable_set('@session', @session)
  @client.instance_variable_set('@current_path', path2)
  @client.instance_variable_set('@start_time', Time.now - 2)
  @session.__next(:add_measured_time) { |pth, measure|
    assert_equal(path2, pth)
    assert_instance_of(TimeMeasure, measure)
  }
  @session.__next(:name_tree) { @tree }
  time_measure = @client.stop
  assert_equal([path], @client.suspend_tasks)
  assert_instance_of(TimeMeasure, time_measure)
  assert_equal(2, time_measure.duration)
end
def test_stop_with_suspend_tasks
  assert_raises(RuntimeError) { @client.stop }
  path = ['Project1', 'Task2', 'Subtask']
  path1 = ['Project1', 'Task2']
  path2 = ['Project1']
  @config.__next(:rdpm_dir) { @datadir }
  @client.instance_variable_set('@suspend_tasks', [path, path1, path2])
  @client.instance_variable_set('@session', @session)
  @client.instance_variable_set('@current_path', path2)
  @client.instance_variable_set('@start_time', Time.now - 2)
  @session.__next(:name_tree) { @tree }
  @session.__next(:add_measured_time) { |pth, measure|
    assert_equal(path1, pth)
    assert_instance_of(TimeMeasure, measure)
  }
  time_measure = @client.stop(path1)
  assert_equal([path, path1], @client.suspend_tasks)
  assert_instance_of(TimeMeasure, time_measure)
  assert_equal(2, time_measure.duration)
end
def test_stop_then_start_previous_task
  assert_raises(RuntimeError) { @client.stop }
  path = ['Project1', 'Task2']
  path2 = ['Project1']
  @config.__next(:rdpm_dir) { @datadir }
  @client.instance_variable_set('@suspend_tasks', [path, path2])
  @client.instance_variable_set('@session', @session)
  @client.instance_variable_set('@current_path', path2)
  @client.instance_variable_set('@start_time', Time.now - 2)
  @session.__next(:add_measured_time) { |pth, measure|
    assert_equal(path2, pth)
    assert_instance_of(TimeMeasure, measure)
  }
  @session.__next(:name_tree) { @tree }
  time_measure = @client.stop
  assert_equal([path], @client.suspend_tasks)
  assert_instance_of(TimeMeasure, time_measure)
  assert_equal(2, time_measure.duration)
  assert_equal(true, @client.started?)
end
```

```
def test_stop_with_queue
  assert_raises(RuntimeError) { @client.stop }
  path = ['Project1', 'Task2']
  @config.__next(:rdpm_dir) { @datadir }
  @client.instance_variable_set('@session', @session)
  @client.instance_variable_set('@current_path', path)
  @client.instance_variable_set('@start_time', Time.now - 2)
  @session.__next(:add_measured_time) { |pth, measure|
    raise DRb::DRbConnError
  }
  time_measure = @client.stop
  assert_instance_of(TimeMeasure, time_measure)
  assert_equal(2, time_measure.duration)
  assert_equal([path, time_measure], @client.queue)
  assert_nil(@client.current_path)
  assert_nil(@client.start_time)
end
def test_synchronize
  @client.instance_variable_set('@session', @session)
  queue = [[['Project1'], 'time_measure2'], \
    [['Project1', 'Task2'], 'time_measure']]
  FileUtils.mkdir_p(@datadir)
  file_path = File.join(@datadir, 'queue.yaml')
  File.open(file_path, 'w') { |fh|
    fh.puts queue.to_yaml
  }
  @config.__next(:rdpm_dir) { @datadir }
  @session.__next(:add_measured_time) { |path, time|
    assert_equal('time_measure2', time)
    assert_equal(['Project1'], path)
  }
  @session.__next(:add_measured_time) { |path, time|
    assert_equal('time_measure', time)
    assert_equal(['Project1', 'Task2'], path)
  }
  @client.synchronize
  assert(@client.queue.empty?)
end
```

File: client.rb

```
attr_reader :current_path, :start_time, :last_path, :queue, :suspend_tasks
def initialize(config)
  @config = config
  @current_path = nil
  @last_path = []
  @queue = []
  @suspend_tasks = []
end
def internal_start(path)
  check_path(path)
  ::Kernel.at_exit {
    internal_stop() rescue RuntimeError
  }
  @suspend_tasks.include?(path) ? nil : @suspend_tasks.push(path)
  @last_path = @suspend_tasks.last
  @current_path = path
  @start_time = Time.now
end
def internal_stop(path=nil)
  if(path)
    check_path(path)
    @current_path = path
    @suspend_tasks.pop
  end
```



```
end
unless(started?)
  raise 'No started task.'
end
begin
  time_measure = TimeMeasure.new(@start_time, Time.now)
  connection {
    @session.add_measured_time(@current_path, time_measure)
  }
rescue ConnectionError
  @queue.push([@current_path, time_measure])
  file_path = create_file_path('queue.yaml')
  File.exist?(file_path) ? File.delete(file_path) : nil
  File.open(file_path, 'w') { |fh|
    fh.puts @queue.to_yaml
  }
end
@current_path = @start_time = nil
time_measure
end
def start(path)
  if(started?)
    internal_stop()
  end
  internal_start(path)
end
def started?
  !@current_path.nil? && !@start_time.nil?
end
def stop(path=nil)
  if(path.nil?)
    @suspend_tasks.pop
  end
  time_measure = internal_stop(path)
  if(!@suspend_tasks.empty?)
    @last_path = @suspend_tasks.last
    internal_start(@suspend_tasks.last)
  end
  time_measure
end
def stop_all
  @suspend_tasks = []
  @current_path = @last_path
  internal_stop()
end
def synchronize
  file_path = File.join(@config.rdpd_dir, 'queue.yaml')
  if(File.exist?(file_path))
    queue = YAML::load(File.read(file_path))
    queue.each { |tm|
      @session.add_measured_time(tm.first, tm.at(1))
    }
    @queue = []
  end
end
```

File. test_commandline.rb

```
class ClientStub
  attr_writer :ping, :whoami, :users, :login, :logged_in, \
    :create_user, :check_path
  attr_accessor :name_tree, :last_path, :suspend_tasks, \
    :current_path
  attr_reader :synchronize
end
```

```
...
...
end
def test_status
  @client.last_path = ["foo", "bar"]
  @client.suspend_tasks = [["foo"], ["foo", "bar"]]
  @client.current_path = ["foo", "bar"]
  assert_equal("status: working on /foo/bar", @cli.status())
  @client.suspend_tasks = []
  assert_equal("status: pausing on /foo/bar", @cli.status())
  @client.last_path = []
  assert_equal("status: stopped", @cli.status())
end
```

File: commandline.rb

```
def start(str_path)
  str_path.nil? ? str_path = format_path(@client.last_path) : str_path
  @client.start(parse_path(str_path))
end
def status
  last_path = @client.last_path
  if(@client.suspend_tasks.empty? && !last_path.empty?)
    "status: pausing on #{format_path(last_path)}"
  elsif(!last_path.empty?)
    "status: working on #{format_path(@client.current_path)}"
  else
    "status: stopped"
  end
end
def stop(str_path)
  path = str_path ? parse_path(str_path) : nil
  @client.stop(path).duration
end
def _run
  #Readline.completer_word_break_characters = ''
  #Readline.completion_proc = self.method(:complete)
  while(line = Readline.readline(prompt(), true))
    # attempt to connect once per command if configured to do so
    login_retry = true
    command, args = parse_command(line)
    if(['quit', 'exit'].include?(command))
      puts('Goodbye')
      break
    elsif(!command.empty?)
      begin
        puts dispatch(command, args)
      rescue RangeError, ConnectionError => @last_error
        if(@config.autologin && login_retry)
          login_retry = false
          _autologin(@autologin_server)
          retry
        end
        @whoami = nil
        puts "Not Connected"
      rescue Exception => @last_error
        puts @last_error.message
      end
      $stdout.puts status()
    end
  end
end
```

```
def dispatch(command, args)
  .....
  .....
  when 'st', 'stop'
    if(args.first == "all")
      format_time(@client.stop_all.duration)
    else
      format_time(stop(args.first))
    end
  when 'users'
    @client.users.sort.join(', ')
  when 'whoami'
    @client.whoami()
  else
    raise "Unknown Command: '#{command}'"
  end
end
def _autologin(server)
  unless(logged_in?)
    login(server)
    @client.synchronize
  end
  rescue DRb::DRbConnError, RRBA::AuthenticationError
end
```